

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour

Total Marks:50

Annexure A

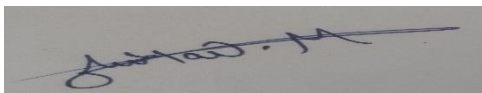
Scenario Based Assignment

Code of Conduct:

I M.GOKUL 192565067__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Scenario Based Assignment

Student 14: Gokul M | Register Number: 192565067

1. Scenario

An airport is designing an AI system to automatically assign arriving flights to available gates. The system must satisfy multiple constraints: no two flights can occupy the same gate at overlapping times, a limited number of gates support wide-body aircraft, certain airlines have contractual gate preferences, ground crew availability constraints, and special accommodations for delayed connecting flights. The complexity increases as the number of flights and gates grows.

Tasks:

Formulate the problem as a Constraint Satisfaction Problem (CSP).

- i. Clearly define variables, domains, and constraints with explanation.
- ii. Explain how backtracking search works in this scenario.
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency.
- iv. Analyze real-world challenges and justify the best strategy for scalability.

2. Scenario

A board-game company is developing an AI opponent for a turn-based strategy game similar to checkers, to be deployed on low-power tablets. The AI must compete against human players, make decisions within strict time limits, evaluate complex board states with multiple

possible moves, and the decision tree is extremely large, making computation expensive on limited hardware.

Tasks:

- i. Explain how the Minimax algorithm models this problem.
- ii. Analyze the computational challenges of large game trees.
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.
- iv. Design an evaluation function and justify the factors considered.
- v. Recommend a strategy for balancing optimality and real-time performance.

3. Scenario

A city's emergency dispatch center uses an AI routing system to send ambulances to accident sites during peak-hour traffic. The environment is highly dynamic - roads may suddenly close due to accidents or construction, and travel cost varies with congestion. The dispatch system has access to: a heuristic estimate (straight-line distance to the incident), partial real-time updates about road closures, and variable travel costs depending on traffic and time of day. Because every second counts, the system must quickly decide the ambulance's route with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty.
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic.
- iii. Analyze the impact of heuristic accuracy on both algorithms.
- iv. Discuss how dynamic changes (sudden road closures) affect search performance.
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues

Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Scenario: Airport Flight Gate Assignment using Constraint Satisfaction Problem (CSP)

Scenario

An airport is developing an Artificial Intelligence (AI) system to automatically assign arriving flights to available airport gates. The system must satisfy several operational constraints, such as preventing gate conflicts, assigning wide-body aircraft only to compatible gates, following airline gate preferences, considering ground crew availability, and accommodating delayed connecting flights. As the number of flights and gates increases, the scheduling problem becomes highly complex. Constraint Satisfaction Problem (CSP) techniques are used to efficiently assign gates while satisfying all constraints.

i. Formulate the problem as a Constraint Satisfaction Problem (CSP)

Variables

Variables represent the flights that require gate assignments.

Example:

- $F1 = \text{Flight } 101$
- $F2 = \text{Flight } 102$
- $F3 = \text{Flight } 103$
- ...
- $F_n = \text{Flight } n$

Each variable corresponds to one arriving flight.

Domains

The domain represents all possible gates that each flight can be assigned.

Example:

- $F1 \rightarrow \{G1, G2, G3, G4\}$
- $F2 \rightarrow \{G1, G2, G3\}$
- $F3 \rightarrow \{G2, G4\}$

Wide-body aircraft are limited to compatible gates.

Constraints

1. Gate Occupancy Constraint

Two flights cannot occupy the same gate during overlapping time periods.

Example:

If F1 and F2 arrive simultaneously,

$F1 \neq F2$ (same gate not allowed)

2. Aircraft Compatibility Constraint

Wide-body aircraft must be assigned only to wide-body compatible gates.

Example:

Wide-body flight

Domain = {G1, G3}

3. Airline Preference Constraint

Certain airlines prefer specific gates.

Example:

Airline A $\rightarrow$ Gate G2 preferred

4. Ground Crew Constraint

Only gates with available ground staff can be assigned.

5. Connecting Flight Constraint

Delayed connecting flights should be assigned nearby gates to reduce passenger transfer time.

ii. Explain how Backtracking Search works

Backtracking is a trial-and-error search algorithm.

Steps

1. Assign the first flight to an available gate.
2. Check whether all constraints are satisfied.
3. If valid, assign the next flight.
4. Continue until every flight is assigned.
5. If a conflict occurs, return to the previous assignment.
6. Try another gate.
7. Repeat until a valid assignment is found.

Example

F1 → G1 ✓

F2 → G1 ✗

Conflict

Backtrack

F2 → G2 ✓

Continue...

Backtracking guarantees finding a solution if one exists.

iii. Explain Constraint Propagation (Forward Checking)

Forward Checking reduces unnecessary searching.

Whenever a gate is assigned,

- Remove that gate from the domains of conflicting flights.
- Detect conflicts early.
- Prevent invalid assignments.

Example

Before assignment

$F1 = \{G1, G2\}$

$F2 = \{G1, G2, G3\}$

Assign

$F1 \rightarrow G1$

Updated domain

$F2 = \{G2, G3\}$

Since G1 is already occupied, it is removed.

Advantages

- Reduces search space
- Detects failures early
- Faster scheduling
- Fewer backtracking steps

iv. Real-world Challenges and Best Strategy

Challenges

- Hundreds of flights every day
- Flight delays
- Emergency landings
- Gate maintenance
- Limited wide-body gates
- Dynamic schedule changes
- Weather disruptions

Best Strategy

The best approach is:

Backtracking + Forward Checking + Constraint Propagation + Heuristics

Using heuristics such as:

- Minimum Remaining Values (MRV)
- Degree Heuristic
- Least Constraining Value (LCV)

Benefits:

- Faster assignment
- Less computation
- Handles large airports
- Scalable
- Produces near-optimal gate allocation

Conclusion

Constraint Satisfaction Problem (CSP) is an effective approach for airport gate assignment. Backtracking ensures valid assignments, while Forward Checking and heuristic methods significantly improve efficiency, making the system suitable for real-world airport operations.

2. Scenario: AI Opponent for Turn-Based Strategy Game using Minimax

Scenario

A board-game company is developing an AI opponent for a turn-based strategy game similar to checkers. The AI runs on low-power tablets and must make intelligent decisions within strict time limits. Since the number of possible game states is extremely large, efficient search techniques are required.

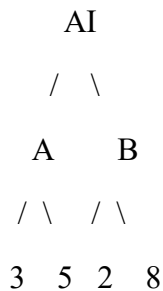
i. Explain how Minimax models this problem

Minimax assumes:

- AI is the maximizing player.
- Human is the minimizing player.

The algorithm explores every possible move.

Decision tree example



Human chooses the minimum value.

AI chooses the maximum among them.

Final decision:

$A \rightarrow 3$

$B \rightarrow 2$

AI chooses 3

Thus AI selects the move that gives the best guaranteed outcome.

ii. Computational Challenges

Large games produce enormous search trees.

Problems include:

- Millions of possible positions
- High memory usage
- Slow computation
- Limited tablet processor
- Strict response time

Complexity:

$O(b^d)$

b = branching factor

d = search depth

As depth increases, computation grows exponentially.

iii. Explain Alpha-Beta Pruning

Alpha-Beta Pruning removes branches that cannot influence the final decision.

Two values are maintained:

Alpha = Best score for AI

Beta = Best score for opponent

If

$\text{Alpha} \geq \text{Beta}$

Remaining branches are ignored.

Example

MAX

/ \

5 MIN

/ \

3 9

After finding 3,

the value 9 is ignored because it cannot improve the decision.

Advantages

- Faster search
 - Same optimal result as Minimax
 - Less memory
 - More suitable for real-time games
-

iv. Design an Evaluation Function

Evaluation Score

Score =

$(5 \times \text{Pieces})$

+

$(3 \times \text{Kings})$

+

$(2 \times \text{Mobility})$

+

$(2 \times \text{Board Control})$

-

$(4 \times \text{Opponent Pieces})$

Factors

Number of pieces

More pieces increase winning chances.

Kings

Kings have greater movement.

Mobility

More legal moves provide flexibility.

Board Control

Central positions offer tactical advantages.

Opponent Strength

More opponent pieces reduce the score.

v. Best Strategy

Recommended approach:

- Minimax
- Alpha-Beta Pruning
- Depth-limited search
- Good evaluation function
- Move ordering

Benefits

- Fast decisions
 - Lower memory usage
 - High-quality gameplay
 - Suitable for low-power tablets
-

Conclusion

Minimax combined with Alpha-Beta Pruning provides an excellent balance between intelligent gameplay and computational efficiency, making it ideal for turn-based games on limited hardware.

3. Scenario: Ambulance Routing using Greedy Best-First Search and A*

Scenario

A city's emergency dispatch center uses an AI routing system to send ambulances to accident locations. The road network changes dynamically due to traffic congestion, accidents, and road closures. The AI must quickly compute the safest and fastest route using heuristic information and real-time traffic updates.

i. Explain Greedy Best-First Search

Greedy Best-First Search selects the node with the smallest estimated distance to the destination.

Formula

$$f(n) = h(n)$$

where

- $h(n)$ = heuristic estimate

Strengths

- Very fast
- Low memory usage
- Suitable for emergencies
- Quickly reaches the destination

Weaknesses

- Ignores actual travel cost
 - May choose congested roads
 - May produce non-optimal routes
 - Sensitive to inaccurate heuristics
-

ii. Explain A* Search

A* considers both actual cost and heuristic estimate.

Formula

$$f(n) = g(n) + h(n)$$

where

- $g(n)$ = travel cost from start
- $h(n)$ = estimated cost to goal

A* balances

- shortest distance
- traffic conditions
- estimated remaining distance

Thus, it usually finds the optimal route.

iii. Impact of Heuristic Accuracy

Accurate Heuristic

- Faster search
- Fewer explored roads
- Better routes

Poor Heuristic

- More unnecessary exploration
- Slower computation
- Less efficient routing

For A*, an admissible heuristic guarantees optimal solutions.

iv. Effect of Dynamic Road Closures

Sudden changes require the search to update.

Problems include:

- Previously selected routes become unavailable
- Increased travel time
- Need for rerouting
- Additional computation

The AI must continuously update road information and recompute the best path when changes occur.

v. Best Recommendation

The recommended algorithm is **A***.

Reasons

- Considers actual travel cost and estimated distance.
- Produces optimal or near-optimal routes.
- Adapts better to traffic information.
- Improves ambulance response time.
- Enhances passenger and patient safety.
- Suitable for dynamic environments with real-time updates.

For highly dynamic road networks, A* can be combined with real-time replanning techniques to quickly adapt to changing traffic conditions.

Conclusion

While Greedy Best-First Search is fast, it may select inefficient routes because it ignores actual travel costs. A* Search provides a better balance between speed and optimality, making it the most suitable algorithm for emergency ambulance routing in dynamic city environments.